

* SOLID Principles :-

Date.....

Sagar 02/4

→ Why Design Principles?

As projects grow into thousands of classes, poor architecture leads to tightly coupled code. This creates "spaghetti code" where bugs are hard to find, new features are difficult to add, and system becomes impossible to maintain.

→ The Goal :- To create clean, scalable, and maintainable software architecture that reduces technical debt and monetary loss.

S → Single Responsibility Principle (SRP)

O → Open/Closed Principle (OCP)

L → Liskov Substitution Principle (LSP)

I → Interface Segregation Principle (ISP)

D → Dependency Inversion Principle (DIP)

* Single Responsibility Principle (SRP) :-

→ Definition : A class should have only one reason to change. It should do only one thing.

→ Problem : A 'ShoppingCart' class that calculates total price, prints the invoice, and saves data to a database. If you change the database logic, you have to modify

the 'ShoppingCart' class. This violates SRP.

→ Solution: Use Composition. Break the monolithic class into smaller, specialized classes:

- ShoppingCart → Only handles product management & total calculation.
- InvoicePrinter → Handles printing logic.
- DBStorage → Handles database persistence logic.

→ SRP doesn't mean a class can only have one method; it means all methods in the class must work toward a single, unified purpose.

* Open/Closed Principle (OCP):-

→ Definition: A class should be open for extension but closed for modifications.

This means you should be able to add new functionality (extension) without changing the existing code of the class (modification).

→ Problem: Suppose you have a 'DBStorage' class.

If you want to add support for a MongoDB or FileSystem storage, you might have tempted to add new methods like 'saveToMongo()' and 'saveToFile()' to the existing class. This violates OCP because you are modifying a tested functional class.

- Solution: • Instead of modifying the existing class, you introduce an Interface (or Abstract class) called 'Persistence'.
- You then separately create concrete classes for each storage type (e.g. SQL Persistence, Mongo Persistence, File Persistence) that implement the Persistence Interface.
 - When a new storage type (like Cassandra) is needed later, you simply create a new class implementing the interface, leaving the original 'ShoppingCart' and existing 'Persistence' logic untouched.

* Liskov Substitution Principle (LSP):-

→ Definition: Subclasses should be substitutable for their base classes.

If you have a base class, you should be able to replace it with any of its derived classes without breaking the functionality of your application.

→ Problem: Imagine a 'BankingAccount' class with deposit() and withdraw() methods. If you create a 'FixedDepositAccount' that inherits from it but cannot perform a withdraw() [due to banking rules], you are forced to throw an exception if someone calls that method.

This violates LSP because the client code, which expects every Account to support withdrawal, will break when it encounters a 'FixedDepositAccount'.

→ Solution: • Instead of forcing a ~~faulty~~ faulty inheritance, break the hierarchy into smaller, logical base classes:

→ Create a 'DepositOnlyAccount' base class.

→ Create a 'WithdrawableAccount' that inherits from DepositOnly and adds the 'withdraw()' method.

- Now, FixedDepositAccount only inherits from DepositOnlyAccount, and your client code can clearly distinguish between accounts that support withdrawals and those that don't.

To adhere to LSP, subclasses must follow three specific rules: →

1. Signature Rule: This rule ensures that the interface of subclass remains compatible with the expectations set by the parent.

- Method Argument Rule → When overriding a method, the subclass should accept arguments of same type or a broader type than the parent (contravariance). This ensures the client, expecting parent's contract, won't face errors.

Date.....

- Return Type Rule → The subclass must return either the same type or a narrower (subclass) type than the parent (covariance).
- Exception Rule → A subclass method should only throw the same exception as the parent method or a narrower (more specific) exception. Throwing a broader or entirely different exception could break the client's try-catch logic.

2. Property Rule: Subclasses must maintain the state-based guarantees established by parent class.

- Class Invariant → Any rule that is always true for the parent (e.g. "balance cannot be negative") must remain true for the subclass.
- History Constraint → A subclass should not allow state changes that were disallowed by the parent. For example, if a parent's account allows withdrawals, a subclass (like FD account) should not override the method to block that behaviour unexpectedly.

3. Method Rule : This governs the behaviour of overridden methods regarding their conditions.

- Pre-conditions → The subclass can relax the conditions for running a method (make them easier to satisfy) but cannot make them stricter.
- Post-conditions → The subclass must ensure that any result or state change promised by the parent's method still holds true after the subclass method executes. It can strengthen post-conditions but never weaken them.

* Interface Segregation Principle (ISP) :-

→ Definition : Clients should not be forced to depend on methods they do not use. Instead of one large, "fat" interface, it is better to create multiple, smaller, and more specific interfaces.

→ Problem : If an interface is too generic (like a Shape Interface containing both `area()` and `volume()`), subclasses that don't need all those features (like a Square which is 2D and has no volume) are

Date.....

forced to implement unnecessary methods, often resulting in empty methods or throwing errors.

→ Solution : Segregate the interface into smaller ones. For instance, separate 2D-Shape (with area()) from 3D-Shape (with area() and volume()). This ensures that a Square only inherits from 2D-Shape, and a Cube inherits from 3D-Shape.

* Dependency Inversion Principle :-

→ Definition : High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces). Furthermore, abstractions should not depend on details; details should depend on abstractions.

→ Problem : When a high-level module (e.g. UserService) directly interacts / interacts with a low-level module (e.g. MySQL Database), the system becomes rigid. Changing the database (e.g. moving to MongoDB), requires modifying the high-level code, violating the Open/Closed Principle.

Date.....

Sagar02K4

→ Solution: Introduce an abstraction (an interface like persistence). The UserService will interact only with the persistence interface. Both the MySQL and MongoDB implementations will then implement this interface. This allows the application to remain unchanged regardless of the underlying database technology.